

Basic, Scientific & Unit Calculator

Developer: Sarthak Sandhan

Class : SY-Computer Science Engineering (Diploma) B.tech

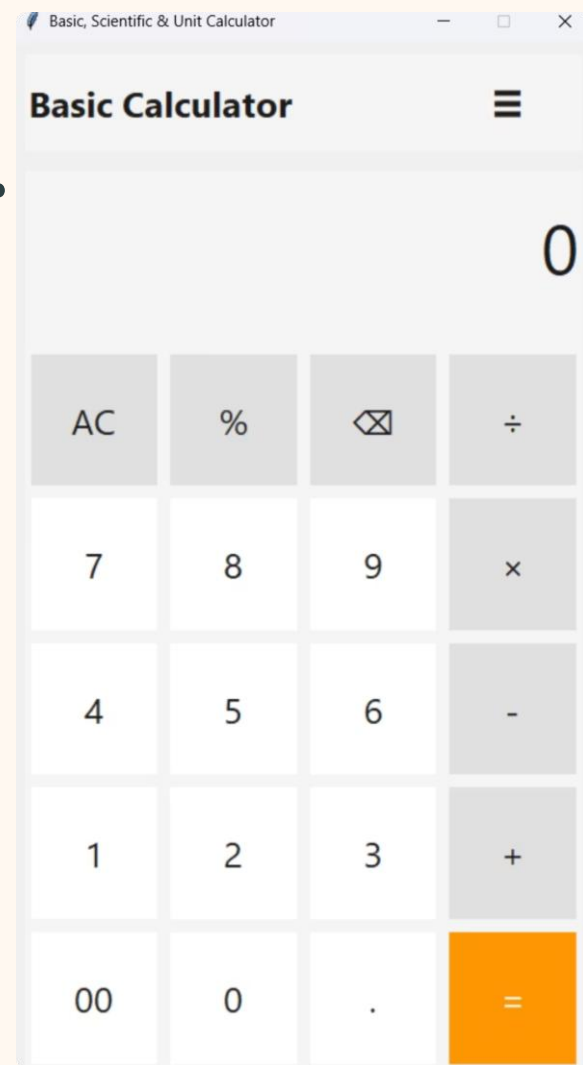
College : Sanjivani University

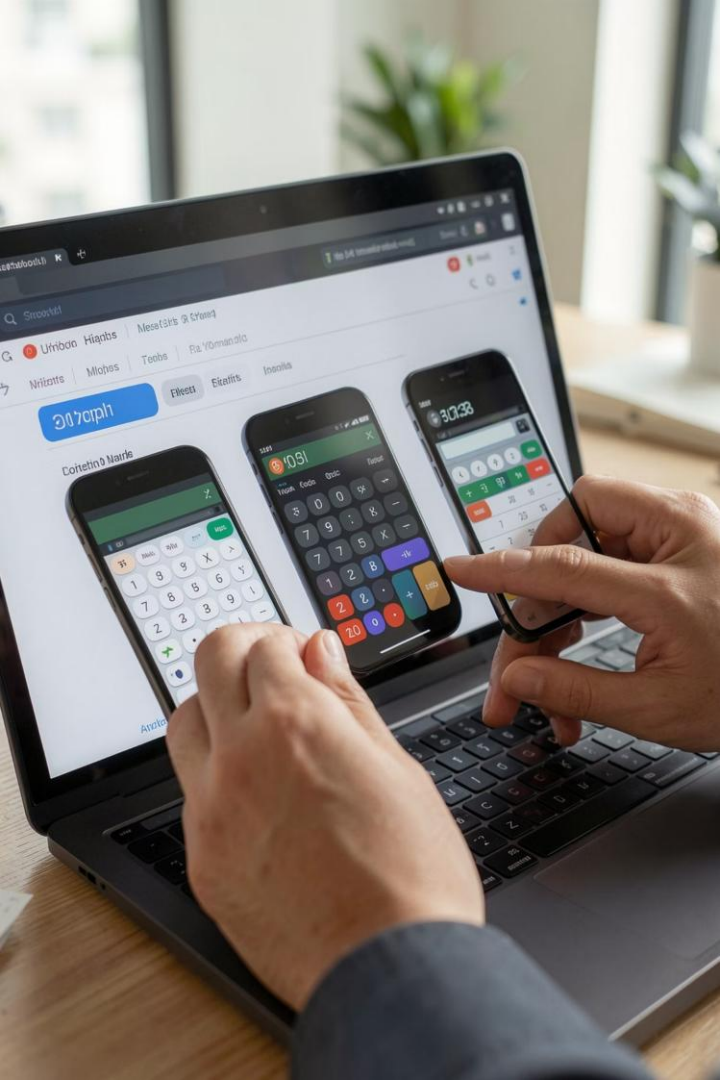
Project Description : A Python Desktop Application for Basic, Scientific, and Unit Calculations.

PYTHON

TKINTER

OOP





Problem Statement

Users often need separate tools for basic calculations, scientific operations, and unit conversions. This increases time, complexity, and reduces productivity.



Time Consumption

Context switching between apps adds friction during study and coding sessions.



Different Interfaces

Different controls and layouts cause errors and slow learning.

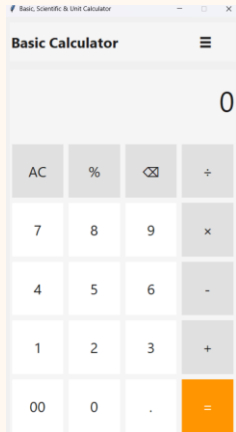


Multiple Tools Required

No single desktop tool combines basic, scientific, and unit conversion needs in one reliable package.

Project Overview

A Python + Tkinter application that combines Basic Calculator, Scientific Calculator, and Unit Converter modules in one interface.



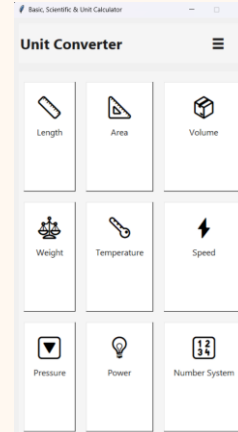
Basic Calculator

Essential arithmetic, percentage, backspace, and a clean numeric keypad for fast entry.



Scientific Calculator

Trigonometry, logarithms, factorials, radians/degrees — designed for engineering coursework and quick prototyping.



Unit Converter

Comprehensive categories: length, area, volume, weight, temperature, speed, pressure, power, and number systems — easy selection and accurate results.

Technology Stack

Python

Core programming language with simple syntax and extensive libraries.

Tkinter

Built-in GUI toolkit for creating desktop applications.

JSON

Stores conversion data and application configurations.

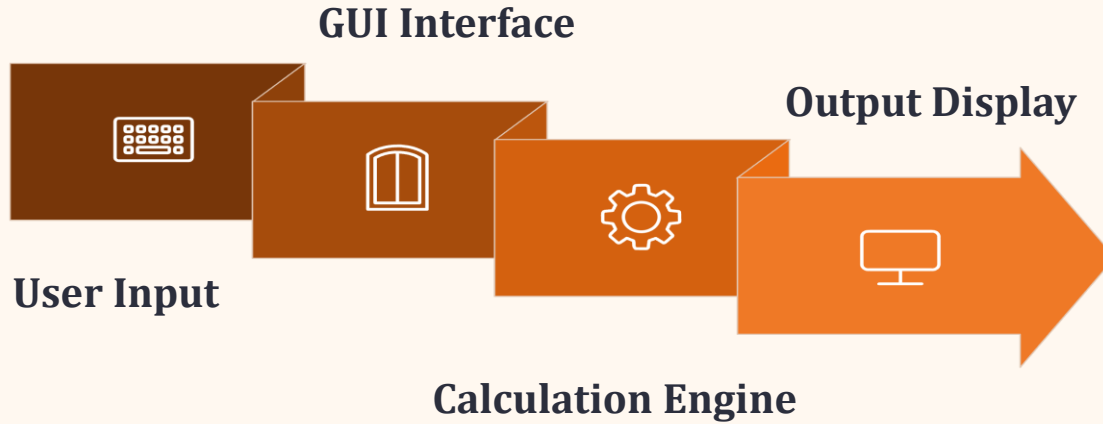
Math Library

Provides scientific functions such as trigonometry, logarithms, and factorials.

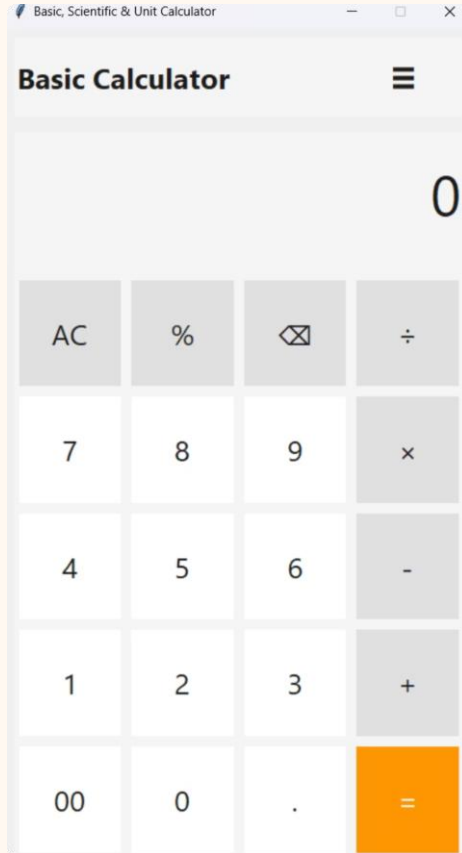
OOP Architecture

Modular classes separate the GUI, calculation engine, and converter logic for better maintainability.

System Architecture



The architecture separates the user interface, calculation logic, and conversion services, making the application modular and easy to maintain.



Basic Calculator Features

- Core arithmetic: $+$ $-$ $\times$ $\div$ with a large keypad and keyboard support
- Percentage calculations and double-zero (00) entry for speed
- Backspace / clear (AC) and error-safe input handling
- Simple and user-friendly interface for everyday calculations

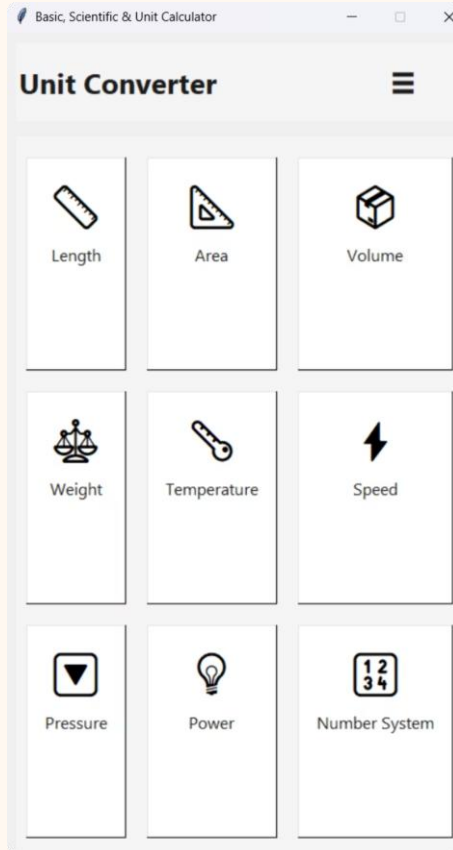
❏ Design emphasizes quick numeric entry and minimal visual clutter — ideal for classroom and interview demonstrations.



Scientific Calculator Features

- Trigonometric functions: sin, cos, tan, with rad/deg toggle.
- Logarithmic functions: log (base 10), ln.
- Advanced operations: x^y , factorial, reciprocal ($1/x$), and π and e constants.
- Floating-point handling and input validation for robust results.

 Mode toggle (rad/deg) and a memory-friendly layout support academic test problems and quick formula verification.



Unit Converter Features

Comprehensive, JSON-driven unit tables enable accurate conversions across common categories:

- Length, area, volume
- Weight, temperature
- Speed, pressure, power
- Number system conversions (binary/decimal/hex)



Extensible: add or update units by editing JSON; conversion functions normalize via base units for consistency.

Demo & Results

Live demo highlights real-time calculations, accurate scientific operations, and fast unit conversions across all modules.



Project Demo Video

Scan to Watch Demo



Project Source Code

Scan to Access Repository



```
File Edit Selection View Go Run Terminal Help
main.py X
main.py > % CalculatorApp > _init_
class CalculatorApp:
    def show_settings(self):
        fg = 'white', bd=2 if self.current_theme == "light" else 0,
        relief=tk.RAISED if self.current_theme == "light" else tk.FLAT,
        cursor="hand2", command=lambda: self.set_theme("light", settings_window))
        light_btn.pack(side=tk.LEFT, padx=10, fill=tk.X, expand=True)

        dark_btn = tk.Button(btn_frame, text="Dark", font=('Segoe UI', 12),
                             bg="#f0f0f0" if self.current_theme == "dark" else theme["bg"],
                             fg="black" if self.current_theme == "dark" else theme["fg"],
                             bd=2 if self.current_theme == "dark" else 0,
                             relief=tk.RAISED if self.current_theme == "dark" else tk.FLAT,
                             cursor="hand2", command=lambda: self.set_theme("dark", settings_window))
        dark_btn.pack(side=tk.LEFT, padx=10, fill=tk.X, expand=True)

        tk.Label(settings_window, text=f"Current: {light if self.current_theme == 'light' else 'Dark'} Mode",
                  font=('Segoe UI', 11), bg=theme["bg"], fg=theme["fg"]).pack(pady=10)

        tk.Label(settings_window, text=f"Decimal Precision: Auto",
                  font=('Segoe UI', 11), bg=theme["bg"], fg=theme["fg"]).pack(pady=5)

        tk.Label(settings_window, text=f"Angle Mode: {self.angle_mode.get().upper()}",
                  font=('Segoe UI', 11), bg=theme["bg"], fg=theme["fg"]).pack(pady=5)

        close_btn = tk.Button(settings_window, text="Close", font=('Segoe UI', 12),
                              bg=theme["bg"], fg=theme["fg"], bd=0, pady=10, cursor="hand2",
                              command=settings_window.destroy)
        close_btn.pack(fill=tk.X, padx=30, pady=20)

    def set_theme(self, theme_name: str, settings_window=None):
        self.current_theme = theme_name
        self.save_theme()
        self.create_main_layout()
        if settings_window:
            settings_window.destroy()

    def clear_content_frame(self):
        for widget in self.content_frame.winfo_children():
            widget.destroy()

    def add_to_history(self, entry: str):
        timestamp = datetime.now().strftime("%H:%M:%S")
        self.history.append(entry.timestamp())
```

```
github.com/sarthaksandhan26-bliy/Project_Two/blob/main/main.py
sarthaksandhan26-bliy / Project_Two
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
Files Project Two / main.py
main README.md main.py
sarthak712 my first upload from vs code
Code Blame 1277 lines (1004 loc) - 46.8 KB
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3 import math
4 import json
5 import os
6 from datetime import datetime
7 from typing import Optional, Dict, Any
8
9 class CalculatorApp:
10     def __init__(self, root):
11         self.root = root
12         self.root.title("Saniar, Scientific & Unit Calculator")
13         self.root.geometry("600x600")
14         self.root.resizable(False, False)
15
16         # Theme variables
17         self.current_theme = "light"
18         self.themes = {
19             "light": {
20                 "bg": "#ffffff",
21                 "display_bg": "#f0f0f0",
22                 "display_fg": "black",
23                 "btn_bg": "white",
24                 "btn_fg": "black",
25                 "btn_bg_fg": "white",
26                 "btn_bg_fg": "black",
27                 "btn_bg_fg": "white",
28                 "btn_bg_fg": "black",
29                 "btn_bg_fg": "white",
30                 "btn_bg_fg": "black",
31                 "border_color": "black"
32             },
33             "dark": {
34                 "bg": "#1a1a1a",
```

Conclusion & Future Scope

Project achievements: integrated three calculators into a single, maintainable desktop app that demonstrates Python proficiency and thoughtful UX. Skills demonstrated include OOP design, GUI development, and data-driven configuration.

1

1. Improve UI/Accessibility

Keyboard-first navigation, higher-contrast modes, and localization support.

2

2. Add Persistence & Settings

Save user preferences, history, and favorite conversions using JSON or SQLite.

3

3. Packaging & Distribution

Build installers (PyInstaller) for Windows/macOS and add CI to publish releases.

Contact Information : Sarthak Sandhan

SY- Computer Science Engineering (Diploma) B.tech

LinkedIn Profile : <https://www.linkedin.com/in/sarthak-sandhan-19b174378/>

Thank You
Questions & Discussion